Nombre de la Institución: Universidad Nacional Rosario castellanos (UNRC)

Licenciatura: Licenciatura en Tecnologías de la Información

Asignatura: Programación Estructurada | Semestre 2

INFORME TÉCNICO: DISEÑO, IMPLEMENTACIÓN Y JUSTIFICACIÓN DEL SISTEMA SIG-CONECTA: GESTIÓN PERSISTENTE DE EQUIPOS EN CONTEXTOS DE BAJA CONECTIVIDAD

Presentado por: Edgar Yair Rosas Flores

Grupo: 213

Profesor: Sergio Martínez

18 de Abril del 2026

1. Introducción

Descripción del Problema Prototípico

En la comunidad de **San Pedro de la Montaña**, la brigada "SIG-Conecta" enfrenta una problemática de alta complejidad técnica y social: administrar un laboratorio comunitario de 20 equipos de cómputo en un entorno con infraestructura eléctrica inestable y conectividad a internet prácticamente nula. Esta condición convierte a cualquier dato almacenado únicamente en la memoria RAM en información vulnerable, ya que un corte de energía no programado provoca la pérdida total del registro de usuarios, incidentes y estado de los equipos.

Relevancia en Contextos de Baja Conectividad

La **Persistencia de Datos** (*Data Persistence*) no es un lujo en este escenario: es el núcleo de la operatividad del sistema. Sin un mecanismo que garantice que la información sobreviva a un reinicio forzado, el administrador del laboratorio se vería obligado a reconstruir manualmente los registros cada sesión, exponiendo el sistema a errores humanos, pérdida de trazabilidad y una gestión inequitativa de los recursos tecnológicos. La solución propuesta —SIG-Conecta v2.0— responde directamente a este incidente crítico mediante el almacenamiento estructurado en archivos de texto plano (.txt), que no requieren conexión a servidores externos ni dependencia de servicios en la nube.

2. Integración de Aprendizajes

¿Cómo contribuyó cada módulo al desarrollo del sistema?

El sistema SIG-Conecta v2.0 es el resultado directo de la aplicación acumulada de los conocimientos adquiridos en los módulos M2, M3 y M4. Cada módulo aportó una capa funcional específica que, integrada, conforma la solución final.

El **Módulo 2** estableció los fundamentos del paradigma de programación estructurada. En él se desarrolló la primera versión del sistema de control de accesos, implementando estructuras de

control secuenciales, selectivas (switch dentro de do-while) y repetitivas (for, while). Se trabajó la gestión del *Input Stream Buffer* mediante cin.clear() y cin.ignore() para prevenir el *Infinite Loop* causado por un *Type Mismatch* —cuando un usuario ingresa texto en un campo que espera un entero—. Este aprendizaje crítico sobre sanitización de entradas (*Input Sanitization*) se integró directamente en todas las funciones de validación del sistema final.

El **Módulo 3** introdujo el paradigma de **Modularización** (*Modular Design*). El código monolítico concentrado en main() fue refactorizado hacia un modelo de alta cohesión (*High Cohesion*), separando la lógica en funciones como visualizarEquipos(), registrarUsuario(), generar_reporte() y consultar_reporte(), cada una con una responsabilidad única. Se dominaron los conceptos de **paso por valor** (*Pass-by-Value*) para funciones de lectura, y **paso por referencia** (*Pass-by-Reference*) con el operador & —aplicado en int &registrosActuales— para funciones que actualizan el estado global del sistema. Esta arquitectura modular fue la base estructural sobre la que se construyó el sistema integrador.

El **Módulo 4** incorporó la pieza central de la solución final: la **Persistencia de Datos** mediante el manejo de flujos de archivos con la librería <fstream>. Se implementaron ofstream para escritura en generar_reporte() e ifstream para lectura en consultar_reporte(). Se introdujo también getline() combinado con stoi() para capturar la opción del menú como texto y convertirla a entero, eliminando los crashes por *Type Mismatch* en la entrada del menú. Asimismo se implementó la función pausarPantalla() con cin.get() para controlar el flujo visual entre operaciones.

¿Qué conceptos fueron clave para resolver el problema?

Los conceptos fundamentales que articularon la solución fueron: la declaración y uso de struct para agrupar datos heterogéneos en una sola entidad lógica, la implementación de arreglos de estructuras para gestionar la bitácora en memoria, el manejo de flujos de archivos (fstream) para la persistencia, y la arquitectura modular para garantizar un código limpio (*Clean Code*), auditable y escalable.

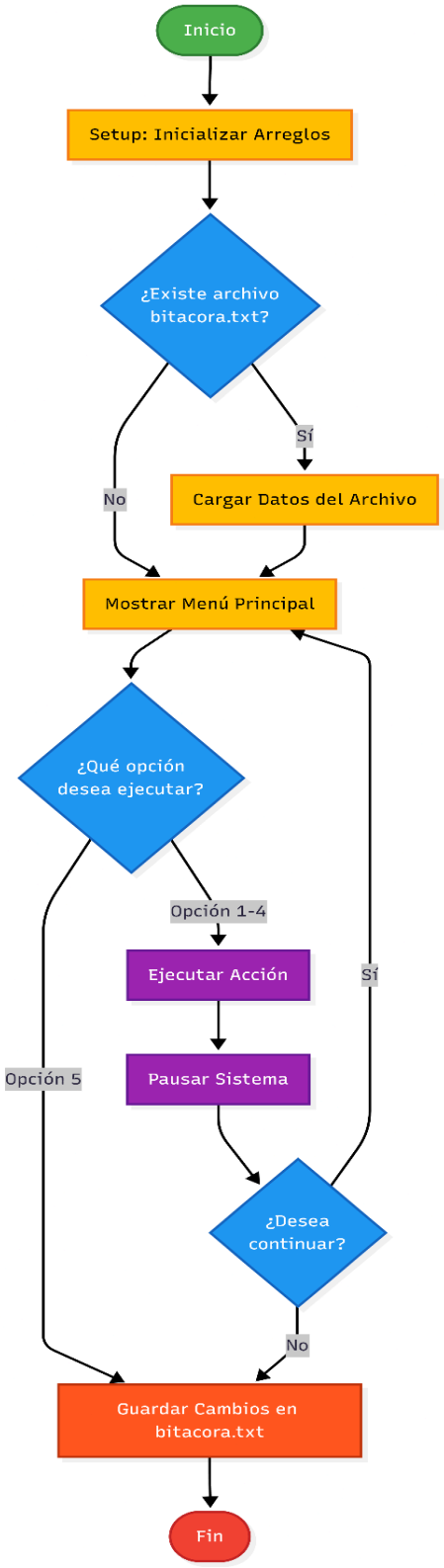
Tabla de integración de aprendizajes por módulo

Concepto	Módulo origen	Función en el sistema final
do-while + switch	M2	main() — ciclo y navegación del menú
cin.fail() + cin.clear() + cin.ignore()	M2	limpiarBuffer() — sanitización en todas las entradas
Funciones modulares	M3	consultarEstado(), modificarEstado(), registrarIncidente(), consultarHistorial()
Paso por referencia &	M3	registrarIncidente() — actualiza totalRegistros en memoria
int equipos[20]	M3	Arreglo global de estados de las 20 computadoras
ofstream / ifstream	M4	guardarBitacora() / cargarBitacora()
getline() para entradas con espacios	M4	Captura de hora, tipoIncidente, descripcion en la struct
pausarPantalla()	M4	Detiene el flujo tras cada operación para facilitar la auditoría
struct RegistroBitacora	AI	Reemplaza string bitacora[10][3] — agrupa 4 campos con tipos correctos
Detección de 4 casos de arranque	AI	cargarBitacora() — distingue primera ejecución, recuperación, archivo limpio y archivo inválido

3. Diseño de la Solución

Diagrama de Flujo del Sistema

-Descripción del diagrama: El flujo inicia con la inicialización de los arreglos en memoria. Inmediatamente se evalúa si existe el archivo bitacora.txt —primera decisión crítica—: si existe, se cargan los datos previos (persistencia de entrada); si no, el sistema arranca en estado limpio. A continuación se despliega el menú principal en un ciclo continuo. Cada selección del usuario (opciones 1 a 4) ejecuta una acción específica y pausa el sistema para que el operador pueda revisar el resultado antes de regresar al menú. La opción 5 rompe el ciclo, ejecuta el guardado en bitacora.txt (persistencia de salida) y termina el programa.



Descripción de la Lógica General

El sistema opera bajo un ciclo principal while controlado por la variable booleana continuar. Al iniciarse, llama a inicializarEquipos() que pone los 20 equipos en estado "Libre" (valor 0 en el arreglo int equipos[20]), y posteriormente a cargarBitacora() que intenta recuperar datos previos del archivo. El menú se gestiona mediante un switch que delega cada opción a su función modular correspondiente. El sistema solo escribe en disco cuando el usuario elige la opción 5, garantizando que ninguna escritura parcial corrompa el archivo.

4. Toma de Decisiones Técnicas

¿Por qué se eligió la combinación de arreglos y estructuras?

Se optó por declarar un struct RegistroBitacora con cuatro campos (idEquipo, hora, tipoIncidente, descripcion) en lugar de una matriz bidimensional de tipo string bitacora[100][4] por una razón técnica de **seguridad de tipos** (*Type Safety*): el campo idEquipo es un entero (int), no una cadena de texto. Almacenarlo como string en una matriz obligaría a realizar conversiones constantes con stoi() o to_string(), aumentando el riesgo de errores en tiempo de ejecución. La struct permite que cada campo tenga su tipo de dato natural, haciendo el código más robusto y semánticamente claro. Para el estado de los equipos se mantuvo el arreglo unidimensional int equipos[20] —heredado del diseño de M4—, ya que su naturaleza numérica (0, 1, 2) es óptima para operaciones de comparación y actualización en ciclos for.

¿Qué decisiones se tomaron para manejar los archivos?

Se tomaron tres decisiones de diseño clave. Primero, el **formato legible por humanos**: en lugar de usar delimitadores como | o comas (formato CSV), se adoptó un esquema de etiquetas CAMPO: valor organizado en dos secciones (--- ESTADOS DE EQUIPOS --- e --- INCIDENTES REGISTRADOS ---). Esto permite que el administrador abra el archivo en el Bloc de Notas y lea la información sin necesitar herramientas adicionales, lo cual es esencial en un entorno rural con recursos limitados.

Segundo, la **persistencia total**: el archivo no solo guarda los incidentes, sino también el estado de los 20 equipos. Esta decisión garantiza que un equipo marcado como "Mantenimiento" conserve ese estado al reiniciar el sistema, evitando asignaciones incorrectas.

Tercero, el **modo de escritura por sobreescritura**: ofstream abre el archivo en modo de reemplazo total. Esto evita la acumulación de datos inconsistentes o duplicados a lo largo de múltiples sesiones, garantizando que bitacora.txt siempre refleje el estado más reciente del

sistema.

¿Qué alternativas se consideraron y por qué se descartaron?

Se evaluó el uso de **archivos binarios** (modo ios::binary), que ofrecen mayor velocidad de lectura y escritura. Sin embargo, fueron descartados porque producen archivos ilegibles en un editor de texto, lo que va en contra del principio de transparencia operativa requerido en el contexto comunitario del proyecto. Se consideró también el uso de **bases de datos SQLite**, pero su implementación requiere librerías externas no disponibles en el entorno de desarrollo básico de la UNRC y supera el alcance del curso de programación estructurada.

5. Evidencia de Funcionamiento

Las siguientes pruebas de caja negra (*Black Box Testing*) verifican el comportamiento del sistema desde la perspectiva del operador, sin analizar el código fuente, únicamente observando entradas y salidas.

Captura 1 — Arranque sin archivo previo (primera ejecución)

Qué debe aparecer:

```
=== Iniciando SIG-Conecta v2.0 ===
[INFO] No se encontro 'bitacora.txt'. Se iniciara registro nuevo.
```

Descripción: Se evidencia el arranque limpio del sistema. Al no existir el archivo de persistencia, `ifstream` no puede abrir ningún canal de lectura, por lo que `cargarBitacora()` detecta el Caso 1 e informa al operador que comenzará un registro nuevo. Todos los equipos se inicializan en estado "Libre" mediante `inicializarEquipos()`.

Captura 2 — Menú principal

Qué debe aparecer:

```
========================================
   SIG-CONECTA | San Pedro de la Montana
========================================
1. Consultar estado de equipos
2. Modificar estado de un equipo
3. Registrar incidente
4. Consultar historial (bitacora)
5. Guardar y salir
========================================
Elige una opcion:
```

Descripción: El menú principal generado por `mostrarMenu()` presenta las cinco operaciones disponibles del sistema. Es invocado al inicio y después de cada operación, garantizando que el operador siempre tenga acceso a todas las funciones sin necesidad de reiniciar el programa.

Captura 3 — Opción 1: Consulta de estado (todos libres)

Qué debe aparecer:

```
========================================
Elige una opcion: 1

--- Estado de los Equipos ---
Equipo 1: Libre
```

```
Equipo 20: Libre

Resumen: 20 libre(s) | 0 ocupada(s) | 0 en mantenimiento
```

Descripción: `consultarEstado()` recorre el arreglo `int equipos[20]` mediante un ciclo `for`. Cada valor entero es traducido a texto legible por `obtenerEstado()`. El resumen final confirma el estado inicial del laboratorio: 20 equipos disponibles al comenzar la sesión.

Captura 4 — Opción 2: Modificar estado (caso de éxito)

Qué debe aparecer:

```
Elige una opcion: 2

--- Modificar Estado de Equipo ---
Numero de equipo a modificar (1-20): 5
Estado actual del Equipo 5: Libre
Nuevo estado:
  1. Libre
  2. Ocupada
  3. Mantenimiento
Seleccione (1-3): 3
[+] Equipo 5 -> Mantenimiento
```

Descripción: `modificarEstado()` implementa doble validación con `while(true)` y `cin.fail()`: primero verifica que el número de equipo esté en rango (1-20), luego que el estado elegido sea válido (1-3). Al confirmar, actualiza el índice correspondiente del arreglo `equipos[4] = 2`.

Captura 5 — Opción 2: Validación de entrada inválida (caja negra de error)

```
Elige una opcion: 2

--- Modificar Estado de Equipo ---
Numero de equipo a modificar (1-20): 25
[ERROR] Numero invalido. Debe ser entre 1 y 20.
Numero de equipo a modificar (1-20): abc
[ERROR] Numero invalido. Debe ser entre 1 y 20.
Numero de equipo a modificar (1-20): 3
Estado actual del Equipo 3: Libre
```

Descripción: Prueba de robustez del sistema. El ciclo `while(true)` combinado con `cin.fail()` y la validación de rango bloquea tanto entradas fuera de rango numérico como entradas alfanuméricas, sin colapsar el sistema ni entrar en bucle infinito. El operador

siempre recibe retroalimentación clara del error.

Captura 6 — Opción 3: Registro de incidente (caso de éxito)

Qué debe aparecer:

```
Elige una opcion: 3

--- Registrar Incidente ---
Equipo afectado (1-20): 5
Hora del incidente (ej. 14:30): 10
Tipo de incidente (ej. Falla hardware, Error software): Falla de hardware
Descripcion breve: La Pantalla no enciende
[+] Incidente registrado. Total en bitacora: 1
```

Descripción: `registrarIncidente()` almacena el incidente en el arreglo de estructuras `bitacora[totalRegistros]`. Los campos de texto se capturan con `getline()`, que admite espacios y caracteres especiales —imposible con `cin >>`—. El campo `idEquipo` se valida previamente con `while(true)` antes de capturar los demás campos.

Captura 7 — Opción 4: Consulta del historial en pantalla

Qué debe aparecer:

```
Elige una opcion: 4

--- Historial de Incidentes ---
No. | Equipo   | Hora  | Tipo                | Descripcion
-------------------------------------------------------------
1   | Equipo 5 | 10 | Falla de hardware | La Pantalla no enciende

Total de registros: 1
```

Descripción: `consultarHistorial()` recorre el arreglo de estructuras `RegistroBitacora` e imprime cada campo en formato de tabla. Verifica primero si `totalRegistros == 0` antes de intentar imprimir, informando al operador con un mensaje claro si la bitácora está vacía.

Captura 8 — Opción 5: Guardar y salir

Qué debe aparecer:

```
5. Guardar y salir
========================================
Elige una opcion: 5
[+] 'bitacora.txt' guardado correctamente (1 incidente(s)).

Sistema cerrado correctamente. Hasta pronto.
```

Descripción : Al seleccionar la opción 5, `guardarBitacora()` abre `bitacora.txt` con `ofstream` en modo sobreescritura, escribe el encabezado del sistema, los estados de los 20 equipos (sección 1) y todos los incidentes del arreglo de estructuras (sección 2), y cierra el flujo con `.close()`. Este cierre explícito garantiza que los datos abandonen la RAM y queden escritos físicamente en el disco.

Captura 9 — Archivo bitacora.txt en el Bloc de Notas

Qué debe aparecer:

```
bitacora.txt
Archivo   Editar   Ver

=====================================================
   BITACORA SIG-CONECTA: San Pedro de la Montana
=====================================================

--- ESTADOS DE EQUIPOS ---
EQUIPO_1: Libre
EQUIPO_2: Libre
EQUIPO_3: Ocupada
EQUIPO_4: Libre
EQUIPO_5: Mantenimiento
EQUIPO_6: Libre
EQUIPO_7: Libre
EQUIPO_8: Libre
EQUIPO_9: Libre
EQUIPO_10: Libre
EQUIPO_11: Libre
EQUIPO_12: Libre
EQUIPO_13: Libre
EQUIPO_14: Libre
EQUIPO_15: Libre
EQUIPO_16: Libre
EQUIPO_17: Libre
EQUIPO_18: Libre
EQUIPO_19: Libre
EQUIPO_20: Libre

--- INCIDENTES REGISTRADOS ---

EQUIPO: 5
HORA: 10
TIPO: Falla de hardware
DESCRIPCION: La Pantalla no enciende
-----------------------------------------------------

Total de incidentes: 1
```

Descripción para el informe: Evidencia de persistencia física. El archivo generado por `ofstream` es texto plano legible en cualquier editor sin herramientas especiales, cumpliendo el principio de transparencia operativa del proyecto. Se observan las dos secciones: el Equipo 5 en estado "Mantenimiento" y el incidente registrado durante la sesión.

Captura 10 — Recuperación de sesión al reiniciar

Qué debe aparecer al arrancar:

```
=== Iniciando SIG-Conecta v2.0 ===
[+] Sesion anterior recuperada: 1 incidente(s) cargados desde 'bitacora.txt'.
```

Y en opción 1:

```
Elige una opcion: 1

--- Estado de los Equipos ---
Equipo 1: Libre
Equipo 2: Libre
Equipo 3: Ocupada
Equipo 4: Libre
Equipo 5: Mantenimiento
```

Descripción: Prueba de resiliencia del sistema —la más importante de la batería—. Al reiniciar, `cargarBitacora()` detecta el Caso 2: archivo válido con datos previos. Parsea ambas secciones del archivo usando `find()` y `substr()`, reconstruye el arreglo `int equipos[20]` y el arreglo de estructuras `RegistroBitacora` en memoria. El Equipo 5 conserva su estado "Mantenimiento" sin intervención manual, demostrando que la persistencia sobrevive al cierre del programa.

6. Análisis de Código: Manejo de Archivos con <fstream>

La librería <fstream>

La librería <fstream> (*File Stream*) es el componente de la biblioteca estándar de C++ que permite establecer canales de comunicación entre el programa y el sistema de archivos del disco duro. Provee dos clases fundamentales: ofstream (*Output File Stream*) para escritura y ifstream (*Input File Stream*) para lectura.

Ciclo de vida de un archivo: Apertura, escritura y cierre

Apertura: Al declarar ofstream archivo("bitacora.txt"), el sistema operativo busca o crea el archivo en el directorio de trabajo. Si el archivo ya existe, lo sobreescribe desde cero. La función

archivo.is_open() verifica que el canal fue establecido correctamente antes de intentar escribir, previniendo errores críticos de ejecución.

cpp

```cpp
ofstream archivo("bitacora.txt");
if (!archivo.is_open()) {
    cout << "[-] ERROR CRITICO: No se pudo crear 'bitacora.txt'.\n";
    return;
}
```

Escritura: El operador << redirige texto desde el programa hacia el archivo, funcionando de manera análoga a cout pero con destino en disco en lugar de la terminal. Se recorre el arreglo de estructuras con un ciclo for para escribir cada registro:

cpp

```cpp
for (int i = 0; i < totalRegistros; i++) {
    archivo << "EQUIPO: "      << bitacora[i].idEquipo      << "\n";
    archivo << "HORA: "        << bitacora[i].hora          << "\n";
    archivo << "TIPO: "        << bitacora[i].tipoIncidente << "\n";
    archivo << "DESCRIPCION: " << bitacora[i].descripcion   << "\n";
}
```

Cierre: La instrucción archivo.close() es crítica: vacía el *buffer* interno del flujo y garantiza que todos los datos sean escritos físicamente en el disco. Sin este cierre explícito, existe riesgo de que datos en el *buffer* se pierdan si el programa termina abruptamente.

La librería <string>

La librería <string> provee el tipo de dato string y la función getline(), indispensables para este sistema. getline(cin, variable) captura una línea completa incluyendo espacios —a diferencia de

cin >> que detiene la captura en el primer espacio—. En la lectura del archivo, getline(archivo, linea) recupera cada línea completa para su posterior parseo con find() y substr(). Sin <string>, sería imposible manejar los campos de texto de la struct RegistroBitacora de forma segura y flexible.

Justificación de las librerías

Ambas librerías son indispensables y complementarias: <fstream> maneja el canal físico hacia el disco y <string> gestiona el contenido textual que viaja por ese canal. Intentar prescindir de <fstream> significaría que todos los datos se perderían al cerrar el programa; prescindir de <string> obligaría a usar arreglos de caracteres (char[]) con tamaño fijo, introduciendo vulnerabilidades de desbordamiento de búfer (*buffer overflow*) —un riesgo crítico en ciberseguridad— al no poder controlar dinámicamente la longitud de las cadenas capturadas.

7. Evaluación de la Solución

¿Cómo contribuye el sistema a resolver la problemática planteada?

SIG-Conecta v2.0 aborda directamente los tres vectores de riesgo identificados en San Pedro de la Montaña. Ante los **cortes de energía**, la persistencia en bitacora.txt garantiza que cualquier dato guardado con la opción 5 sobrevive al reinicio. Ante la **falta de conectividad**, el sistema opera completamente offline, sin requerir red, servidor o base de datos externa. Ante el **desorden operativo**, el menú modular con validaciones en cada entrada reduce el margen de error humano y mantiene un historial de incidentes trazable.

¿Qué limitaciones tiene la solución?

La primera limitación es la **capacidad fija de la bitácora**: el arreglo RegistroBitacora bitacora[100] establece un máximo de 100 incidentes por sesión acumulada, lo que puede ser insuficiente en laboratorios con alta rotación. La segunda limitación es la **ausencia de autenticación**: cualquier persona con acceso físico al equipo puede modificar los estados o

registrar incidentes falsos. La tercera es que la **sobreescritura total** del archivo al guardar implica que si el programa falla a mitad de la escritura, el archivo puede quedar corrupto.

¿Cómo podría escalarse o mejorarse?

En una evolución profesional del sistema, se podrían implementar cinco mejoras concretas. Un **módulo de autenticación** con usuario y contraseña protegería el acceso administrativo. La migración a **escritura en modo ios::app** (*append*) permitiría agregar registros al archivo sin sobrescribir el histórico completo. El uso de **memoria dinámica** con vectores (std::vector) eliminaría el límite fijo de la bitácora. La incorporación de una **base de datos SQLite** reemplazaría los archivos de texto por una solución más robusta y consultable. Finalmente, una **interfaz gráfica** con bibliotecas como Qt haría el sistema accesible a operadores sin conocimientos técnicos.

8. Conclusiones

¿Qué aprendí sobre desarrollar soluciones en contextos reales?

El desarrollo de SIG-Conecta a lo largo de cuatro módulos me demostró que la programación no existe en el vacío: cada decisión técnica tiene consecuencias directas en el entorno donde opera el software. Elegir struct sobre una matriz, decidir el formato del archivo, o implementar la sanitización de entradas no fueron elecciones arbitrarias —fueron respuestas a restricciones reales de infraestructura, operación y seguridad. Aprendí que en contextos de vulnerabilidad técnica, la resiliencia del sistema es más valiosa que su complejidad funcional: un programa que guarda correctamente sus datos en un archivo de texto vale más que uno con veinte funciones que pierde todo al cerrarse.

¿Qué habilidades fortalecí durante la UCA?

A lo largo de esta Unidad de Competencia para el Aprendizaje fortalecí cinco habilidades concretas aplicables a mi trayectoria en TI y ciberseguridad. La **modularización** me enseñó a

dividir problemas complejos en subproblemas manejables con responsabilidades únicas, un principio que se aplica directamente en el análisis de arquitecturas de seguridad. El **manejo de archivos** me introdujo al concepto de almacenamiento no volátil (*Non-volatile Storage*) y su rol en la integridad de los datos. La **sanitización de entradas** me sensibilizó sobre uno de los vectores de ataque más comunes en sistemas de software. El **pensamiento algorítmico** me permitió diseñar flujos de control predecibles y auditables. Finalmente, la **documentación técnica** —expresar en lenguaje preciso las decisiones de diseño— es una habilidad que distingue a un profesional de TI de un simple programador.

REFERENCIAS

Aprende a programar. (2023). *Registros (struct). Curso de C++: Programación básica*. https://www.aprendeaprogramar.com/cursos/verApartado.php?id=16007

Deitel, P. J., y Deitel, H. M. (2014). *C++: Cómo programar* (9.ª ed.). Pearson Educación.

Gómez Fuentes, M. C., y Cervantes Ojeda, J. (2013). *Inicialízate en la programación con C++*. Universidad Autónoma Metropolitana. https://www.academia.edu/18804776/Programacion_en_c_

Microsoft. (2024). *Using C++ in Visual Studio Code*. https://code.visualstudio.com/docs/languages/cpp

Pardo, A. (2022). *C++ course: Fundamentals and practice* [Repositorio de software]. GitHub. https://github.com/AdrianPardo99/cpp-course-1

Programación ATS. (28 de diciembre de 2016). *Programación en C++ || Archivos || Escribir en un archivo de texto* [Video]. YouTube. https://www.youtube.com/watch?v=eTBaxFZkjHg

Programación ATS. (30 de diciembre de 2016). *Programación en C++ || Archivos || Lectura de un archivo de texto* [Video]. YouTube. https://www.youtube.com/watch?v=eTBaxFZkjHg

Soulié, J. (2007). *C++ language tutorial*. Cplusplus.com. https://www.cplusplus.com/doc/tutorial/

Universidad Nacional Rosario Castellanos. (2026). *Módulo 2: Fundamentos de programación estructurada*. https://unrc.secihti.mx/plataforma/pluginfile.php/11920/mod_resource/content/9/LTIC2PRO8_Contenido%20nuclear%20M2.docx.pdf

Universidad Nacional Rosario Castellanos. (2026). *Módulo 3: Contexto problemático — SIG-Conecta*.

https://unrc.secihti.mx/plataforma/pluginfile.php/11904/mod_page/content/19/LTIC2PRO8_Contexto%20Problem%C3%A1tico%20%281%29.pdf

Universidad Nacional Rosario Castellanos. (2026). *Módulo 4: Fundamentos de programación estructurada*. https://unrc.secihti.mx/plataforma/pluginfile.php/11932/mod_resource/content/8/LTIC2PRO8_Contenido%20nuclear_M4.pdf

W3Schools. (s.f.). *C++ tutorial*. https://www.w3schools.com/cpp/default.asp